Deep Learning Viva Preparation Guide

Covering All 10 Experiments

MLP • CNN • Transfer Learning • RNN • LSTM • GRU • GAN

EXP 1 — Multilayer Perceptron (MLP)

1. Architecture

> 📌 **Definition:** An MLP is a feedforward artificial neural network with one or more hidden layers between the input and output layer. Every neuron in each layer is connected to every neuron in the next layer (fully connected).

Layer Structure:

- **Input Layer** — receives raw feature vectors (one neuron per feature)
- **Hidden Layer(s)** — applies linear transformation + non-linear activation
- **Output Layer** — produces class probabilities (Softmax) or regression values

Layer	Role / Typical Config
Input Layer	1 neuron per feature; no activation
Hidden Layer 1	64–512 neurons; ReLU / Sigmoid / Tanh
Hidden Layer 2+	Deeper representation; usually same or smaller width
Output Layer	Neurons = num_classes; Softmax for classification

---

2. Forward Pass

For each layer l:

- $z^{(l)} = W^{(l)} \cdot a^{(l-1)} + b^{(l)}$ (linear transformation)
- $a^{(l)} = g(z^{(l)})$ (activation function applied element-wise)

---

3. Activation Functions — Comparison

Function	Formula	Range	Advantages	Disadvantages	Best Use
Sigmoid	1/(1+e^-x)	(0,1)	Smooth gradient; probabilistic output	Vanishing gradient; not zero-centred	Binary output layer
Tanh	(e^x−e^-x)/(e^x+e^-x)	(−1,1)	Zero-centred; stronger gradient than sigmoid	Still suffers vanishing gradient	Hidden layers (RNNs)
ReLU	max(0,x)	[0,∞)	Computationally cheap; no vanishing gradient in +ve region	Dying ReLU problem; not zero-centred	Default for hidden layers
Leaky ReLU	max(αx,x) α≈0.01	(−∞,∞)	Fixes dying ReLU	Extra hyperparameter α	When ReLU neurons die
ELU	x if x>0, α(e^x−1) if x≤0	(−α,∞)	Smooth; negative values help centre	Computationally expensive	Deep networks
Softmax	e^xi / Σe^xj	(0,1) sum=1	Probabilistic output; multi-class	Not used in hidden layers	Multi-class output only

---

4. Loss Functions

- **Cross-Entropy Loss (classification):** $L = -\Sigma\ y_i \log(\hat{y}_i)$
- **Mean Squared Error (regression):** $L = (1/n)\ \Sigma(y-\hat{y})^2$

---

5. Training — Backpropagation & Gradient Descent

1. Compute loss → 2. Compute gradients via chain rule → 3. Update weights: $W \leftarrow W - \eta \cdot \partial L/\partial W$

💡 **Key Point:** Learning rate η controls step size. Too high → diverge; too low → slow convergence.

---

6. Effect of Network Depth

Aspect	Shallow vs Deep Networks
Representational Power	Deep nets learn hierarchical features; shallow nets need exponentially more neurons for same function
Training Difficulty	Deeper nets suffer from vanishing/exploding gradients
Overfitting Risk	More layers → more parameters → higher risk without regularisation
Convergence Speed	Properly initialised deep nets often converge faster on complex tasks
Practical Sweet Spot	2–4 hidden layers sufficient for most tabular/digit tasks

---

7. Digit Classification (MNIST use-case)

- Input: 28×28 = 784 pixels flattened to a 784-d vector
- Typical architecture: 784 → 512 (ReLU) → 256 (ReLU) → 10 (Softmax)
- Achieves ~98% accuracy with proper training

---

8. Advantages & Disadvantages

Advantages	Disadvantages
Universal approximator (any continuous function)	No spatial/temporal feature awareness
Simple to implement and understand	Requires flat input (loses 2D structure in images)
Works well on tabular and structured data	Many parameters → memory intensive
Flexible activation & loss function choice	Prone to overfitting on small datasets
Fast inference once trained	Sensitive to feature scaling and initialisation

---

9. Key Viva Questions — MLP

1. **What is the vanishing gradient problem and how does ReLU solve it?** Gradients shrink exponentially in deep sigmoid/tanh networks. ReLU has gradient=1 for positive inputs, preventing shrinkage.

2. **Why is Softmax used only at the output layer?** Softmax compresses all neuron outputs into a probability distribution, which only makes sense as a final prediction, not as intermediate features.

3. **What is the dying ReLU problem?** Neurons that receive only negative inputs always output 0 and never update their weights. Leaky ReLU fixes this with a small slope for $x<0$.

4. **How does depth affect convergence?** More layers increase representational power but risk vanishing gradients and slower convergence. Batch norm and proper initialisation (He, Xavier) mitigate this.

5. **What is Batch Normalisation and why is it used?** BatchNorm normalises layer inputs to zero mean and unit variance, stabilising training, allowing higher learning rates, and acting as a regulariser.

EXP 2 — Convolutional Neural Network (CNN)

1. Architecture

> 📌 **Definition:** A CNN exploits spatial structure of images using shared-weight convolutional filters, pooling for dimensionality reduction, and fully connected layers for classification.

Standard building blocks in order:

- **Convolutional Layer** — learnable filters slide over input, produce feature maps
- **Activation (ReLU)** — introduces non-linearity
- **Pooling Layer (Max/Avg)** — downsamples spatial dimensions
- **Batch Normalisation (optional)** — stabilises training
- **Flatten** — converts 3D feature maps to 1D vector

- **Fully Connected Layer(s)** — combines features for classification
- **Output (Softmax)** — class probabilities

---

2. Convolutional Layer — Mechanics

- Filter/Kernel: small matrix (e.g. 3×3, 5×5) with learnable weights
- Stride (S): step size when sliding the filter
- Padding (P): zero-padding to control output size
- Output size: `[(W-F+2P)/S]+1` where W=input width, F=filter size
- Each filter detects one feature (edge, texture, colour blob …)
- Multiple filters → multiple feature maps stacked as channels

> 💡 **Key Point:** Parameter sharing: one filter reuses same weights across all spatial positions → huge parameter reduction vs fully connected.

---

3. Pooling Layers

Max Pooling	Average Pooling
Takes maximum value in pool window	Takes average value in pool window
Preserves strongest activation (edge detectors)	Provides smoothed representation
More common; better for classification	Better for dense prediction tasks
Reduces spatial size (e.g. 28×28 → 14×14 with 2×2 pool, stride 2)	Same size reduction mechanism
Reduces overfitting by limiting parameters	Less aggressive feature selection

---

4. Role of Pooling in Reducing Overfitting

- Reduces spatial dimensions → fewer parameters in FC layers
- Provides translation invariance (small shifts don't change prediction)
- Acts as a form of feature selection
- Combined with Dropout further reduces overfitting

---

5. Feature Map Visualisation

What each layer learns (typical progression):

- **Conv Layer 1:** Low-level — edges, colour gradients, simple textures
- **Conv Layer 2:** Mid-level — corners, curves, simple shapes
- **Conv Layer 3+:** High-level — object parts (eyes, wheels, etc.)
- **Final Conv:** Semantic — class-specific patterns

> 💡 **Key Point:** Visualise by passing an image forward and plotting the activation maps of each channel at each convolutional layer using matplotlib.

---

6. Effect of Convolution Parameters on Accuracy

Parameter	Increase Effect	Decrease Effect
Filter Size (F)	Larger receptive field; captures global patterns; more params	Captures fine-grained local features; fewer params
Number of Filters	Richer feature set; higher accuracy potential; risk overfit	Fewer features; faster; may underfit
Stride (S)	Faster computation; smaller output; loses detail	Slower; larger output; more detail preserved
Padding (P)	Preserves border info; same output size as input	Output shrinks; border info lost
Depth (num layers)	Hierarchical features; better accuracy on complex tasks	Simpler model; faster; may underfit complex data

---

7. CNN vs Fully Connected (MLP) — Image Classification

CNN	MLP (Fully Connected)
Preserves spatial structure of images	Flattens image; loses all spatial info
Parameter sharing → fewer weights	Every neuron independently weighted
Translation invariant via pooling	Not inherently invariant

Scales well to large images	Parameter explosion with large images (224×224×3 = 150k inputs)
State-of-the-art on vision tasks	Practical only for small or tabular data
Requires more engineering (architecture design)	Simpler to implement

---

8. Advantages & Disadvantages of CNN

Advantages	Disadvantages
Automatic spatial feature extraction	Needs large labelled datasets
Parameter sharing reduces model size	Computationally expensive (GPU needed)
Translation / partial rotation invariance	Black-box; hard to interpret
Hierarchical feature learning	Sensitive to hyperparameters
Works directly on raw pixels	Poor at capturing long-range dependencies (→ use Transformers)

---

9. Key Viva Questions — CNN

1. **How does a convolutional layer differ from a fully connected layer?** Conv uses shared local filters (sparse connectivity + weight sharing); FC connects every input to every output neuron independently.

2. **What is the receptive field?** The region in the input image that a neuron in a deeper layer 'sees'. Deeper layers have larger receptive fields, enabling global pattern detection.

3. **Why is max pooling preferred over average pooling?** Max pooling retains the most prominent feature in each region, making it more robust to noise. Average pooling can dilute strong activations.

4. **What happens when stride > 1?** The filter skips positions, producing a smaller output feature map, reducing computation but potentially missing detail.

5. **How do you prevent overfitting in CNNs?** Dropout, Batch Normalisation, Data Augmentation (flips, crops, rotations), L2 regularisation, and early stopping.

EXP 3 — Transfer Learning

1. What is Transfer Learning?

📌 **Definition:** Transfer Learning reuses a model trained on a large dataset (e.g. ImageNet) for a different but related task, either as a fixed feature extractor or through fine-tuning selected layers.

---

2. Two Main Strategies

Feature Extraction (Frozen Layers)	Fine-Tuning (Partial / Full)
Pretrained base layers are frozen (weights unchanged)	Some/all pretrained layers are allowed to update
Only the new top classifier is trained	Entire network or specific blocks update on new data
Very fast training; requires little data	Needs more data and more training time
Works when new task is similar to source task	Works when tasks differ more significantly
Risk of underfitting if tasks are very different	Risk of catastrophic forgetting if learning rate too high
Example: use VGG16 features → train SVM or small FC	Example: unfreeze last 2 blocks of ResNet50 + train

---

3. Common Pretrained Models

Model	Depth	Params (M)	ImageNet Top-5 Acc	Key Innovation

VGG16	16	138	92.7%	Deep uniform 3×3 conv blocks
ResNet50	50	25	93.4%	Skip / residual connections
InceptionV3	48	23	93.9%	Inception modules (parallel convs)
MobileNetV2	53	3.4	90.1%	Depthwise separable convs (lightweight)
EfficientNetB0	—	5.3	93.3%	Compound scaling of depth/width/resolution
VGG19	19	143	92.3%	Deeper VGG variant

---

4. How Pretrained Features Help on Small Datasets

- Low-level features (edges, textures) are universal — useful for any visual task
- Pre-learned features act as a rich initialisation, drastically reducing search space
- The model needs far fewer examples to learn the task-specific top layers
- Regularisation effect: frozen layers cannot overfit to small data

> 💡 **Key Point:** Rule of thumb: <1000 samples → freeze all base; 1k–10k → unfreeze top blocks; >10k → fine-tune more layers.

---

5. Training Efficiency

Training from Scratch	Transfer Learning
All weights randomly initialised	Starts from pre-learned representation
Needs large dataset (millions of images)	Works with hundreds to thousands of images
Many epochs to converge	Converges in far fewer epochs
High computational cost	Only top layers updated → fraction of the compute
High risk of overfitting on small data	Frozen layers prevent overfitting

---

6. Advantages & Disadvantages

Advantages	Disadvantages
Dramatically reduces training time and data needs	Pretrained features may not align if tasks are very different
Achieves high accuracy even with small datasets	Frozen layers may limit performance on unique domains
Leverages knowledge from millions of images	Large base models consume significant memory
Reduces overfitting risk	Choice of unfreezing strategy requires expertise
Industry standard for vision and NLP tasks	Licensing / distribution restrictions on some models

---

7. Key Viva Questions — Transfer Learning

1. **What is catastrophic forgetting?** When fine-tuning with a high learning rate, the model overwrites pretrained weights, losing the generalised features. Fixed by using a small learning rate (e.g. 1e-5) for pretrained layers.

2. **When should you freeze vs fine-tune layers?** Freeze when data is limited and task is similar to source. Fine-tune when you have more data or the task differs significantly.

3. **What is domain shift and how does it affect transfer learning?** Domain shift occurs when source and target data distributions differ greatly (e.g. ImageNet photos → medical X-rays). More fine-tuning is needed; sometimes the benefit is limited.

4. **How do you add a custom classification head to a pretrained model?** Remove the original output layer, add GlobalAveragePooling → Dense (ReLU) → Dropout → Dense (Softmax) layers. Compile and train only these layers first.

EXP 4 — CNN vs Transfer Learning

1. Head-to-Head Comparison

Dimension	CNN from Scratch	Transfer Learning
Training Time	Very long (hours–days)	Short (minutes–hours for feature extract)
Data Requirement	Large (10k–1M+ images)	Small (100–10k images)
Accuracy (small data)	Poor (high overfitting)	High (pretrained features generalise)
Accuracy (large data)	Can match / exceed TL	Similar; small margin
Computational Cost	High GPU cost throughout training	Low (only top layers trained)
Flexibility	Full control over architecture	Constrained by base model design
Interpretability	Same difficulty	Same difficulty
Initial Convergence	Slow	Fast (good initialisation)

---

2. Impact of Dataset Size

Small Dataset (<5k images)	Large Dataset (>100k images)
Transfer Learning wins decisively	Both approaches competitive
Scratch model severely overfits	Scratch model can match TL with enough data
Feature extraction alone may suffice	Full fine-tuning often beneficial
Data augmentation critical for both	Data augmentation still helps

---

3. When to Prefer Transfer Learning

- Task domain is close to ImageNet (photos, everyday objects)
- Dataset is small to medium (<50k labelled samples)
- Limited GPU budget or time constraints
- Need rapid prototyping or baseline performance

4. When Training from Scratch Makes Sense

- Highly specialised domain (e.g. satellite imagery, molecular structures)
- Very large proprietary dataset that differs greatly from ImageNet
- Need custom architecture (non-standard input shape, channels, modality)
- Research into novel architectures

---

5. Training Loss & Accuracy Curves

From Scratch: Loss decreases slowly; validation loss diverges from training loss early (overfitting) on small data.

Transfer Learning: Loss drops sharply in first few epochs; train/val curves remain close; faster plateau.

---

6. Key Viva Questions — CNN vs TL

1. **Why does a CNN trained from scratch overfit on small datasets?** It has millions of randomly initialised parameters with no prior knowledge. With limited examples, it memorises training data. TL starts with meaningful representations.

2. **If given 1 million images, would you still use transfer learning?** It depends on domain. If close to ImageNet, TL still helps for faster convergence. If very different (medical, satellite), training from scratch or full fine-tuning of a pretrained model is preferred.

3. **What is the difference between fine-tuning and feature extraction?** Feature extraction: base frozen, only new head trained. Fine-tuning: some/all base layers also updated with a small learning rate on the new task data.

EXP 5 — Performance Analysis of CNN

1. Learning Rate — Effect on Convergence

Too High LR	Too Low LR
Loss oscillates / diverges	Loss decreases very slowly
Overshoots optimal weights	Gets stuck in local minima
Training unstable; NaN loss possible	Very long training time
Solution: reduce LR (LR scheduler / decay)	Solution: increase LR or use adaptive optimizer

> 💡 **Key Point:** Use LR schedulers: StepLR, CosineAnnealingLR, or ReduceLROnPlateau. Optimal LR found via LR range test (cyclical LR).

---

2. Effect of Epochs on Overfitting & Generalisation

Typical behaviour of loss curves over epochs:

- **Phase 1 (early epochs):** Both training and validation loss decrease — model learning
- **Phase 2 (optimal):** Validation loss reaches minimum — best generalisation
- **Phase 3 (overfit):** Training loss keeps decreasing; validation loss starts increasing

> ⚠️ **Note:** Early stopping monitors validation loss and halts training when it stops improving (patience parameter, e.g. 5–10 epochs).

---

3. Training vs Validation Loss Curves — Interpretation

Curve Pattern	Diagnosis	Solution
Both losses high	Underfitting / model too simple	Increase model capacity; more epochs; reduce regularisation
Train loss low, val loss high	Overfitting	Dropout, BatchNorm, data augmentation, reduce model size
Both losses converge close	Good fit	Fine-tune LR; possibly increase capacity slightly
Val loss < train loss	Regularisation too strong / small validation set noise	Reduce dropout; ensure balanced splits

Loss plateaus early	LR too low or wrong optimizer	LR scheduler; switch optimizer
Loss oscillates	LR too high	Reduce LR; gradient clipping

---

4. Hyperparameter Comparison

Hyperparameter	Low Value	High Value	Recommended Range
Learning Rate	Slow convergence	Divergence / oscillation	1e-4 to 1e-2
Batch Size	Noisy gradient (good regulariser)	Stable but poor generalisation	32–256
Dropout Rate	Less regularisation	Too much info lost	0.2–0.5
Number of Filters	Weak feature extraction	Overfitting; slow	32 → 64 → 128 progression
Epochs	Underfitting	Overfitting	Use early stopping
Weight Decay (L2)	Risk overfitting	Underfitting	1e-5 to 1e-3

---

5. Optimizers Comparison

Optimizer	Mechanism	Pros	Cons
SGD	Fixed LR gradient steps	Simple; good generalisation with momentum	Sensitive to LR; slow convergence
SGD+Momentum	Accumulates past gradients	Faster than plain SGD	Still needs manual LR tuning
Adam	Adaptive per-parameter LR (m + v estimates)	Fast convergence; less LR tuning	Can generalise slightly worse than SGD

RMSprop	Divides grad by running average of recent magnitudes	Works well for RNNs	May not converge to sharp minima
AdaGrad	Accumulates all past squared gradients	Good for sparse data	LR shrinks to 0 over time

---

6. Key Viva Questions — CNN Performance

1. **How do you choose the right learning rate?** Use the LR range test: train for a few epochs while increasing LR exponentially. Plot loss vs LR. The ideal LR is just before loss starts increasing. Start with 1e-3 for Adam.

2. **What is the difference between overfitting and underfitting?** Overfitting: model memorises training data, high train accuracy but low val accuracy. Underfitting: model is too simple to capture the pattern, both accuracies are low.

3. **How does batch size affect training?** Smaller batches introduce gradient noise that acts as regularisation and helps escape local minima but slow training. Larger batches are faster but may converge to sharp minima that generalise poorly.

EXP 6 — Object Detection

1. Object Detection vs Image Classification

Image Classification	Object Detection
One label per image	Multiple objects, each with label + bounding box
Output: class probabilities	Output: (class, confidence, x,y,w,h) per detected object
Examples: CNN, ResNet, VGG	Examples: YOLO, SSD, Faster-RCNN, DETR

---

2. Key Concepts

📌 **Definition: Bounding Box** — rectangle defined by (x_center, y_center, width, height) or (x_min, y_min, x_max, y_max) that localises an object.

📌 **Definition: Confidence Score** — probability that a bounding box contains an object of a specific class. High confidence = model is sure; low = uncertain.

📌 **Definition: IoU (Intersection over Union)** — measures overlap between predicted and ground-truth box. IoU = Area_Overlap / Area_Union. Threshold typically 0.5.

---

3. Popular Pretrained Detection Models

Model	Type	Speed	Accuracy	Best For
YOLOv8	One-stage	Very Fast (real-time)	High	Real-time applications
SSD	One-stage	Fast	Medium	Embedded / edge devices
Faster R-CNN	Two-stage	Slow	Very High	High-accuracy offline tasks
RetinaNet	One-stage	Medium	High	Dense object scenes
DETR	Transformer	Slow	High	Research; complex scenes

---

4. One-Stage vs Two-Stage Detectors

One-Stage (e.g. YOLO, SSD)	Two-Stage (e.g. Faster R-CNN)
Directly predicts boxes and classes in one forward pass	Stage 1: Region Proposal Network (RPN) proposes candidate boxes
Faster; suitable for real-time	Stage 2: Classify and refine proposals → more accurate
Lower accuracy on small/dense objects	Slower but more precise
Used in video surveillance, autonomous driving	Used in medical imaging, satellite analysis

---

5. Non-Maximum Suppression (NMS)

After detection, multiple overlapping boxes for the same object are produced. NMS:

- Selects the box with highest confidence
- Suppresses all other boxes with IoU > threshold (e.g. 0.5)
- Repeats until all boxes processed

---

6. Why Detection Fails — Common Cases

- **Small objects:** limited feature resolution in deeper layers
- **Severe occlusion:** part of object hidden → incomplete features
- **Low image quality / blur / low light:** corrupts feature extraction
- **Novel/rare object classes:** not in training distribution
- **Crowded scenes:** overlapping boxes suppressed incorrectly by NMS
- **Extreme aspect ratios:** default anchors don't match

---

7. Effect of Image Quality on Detection

High Quality Image	Low Quality / Degraded Image
Sharp edges → strong conv features	Blur reduces gradient signal in conv layers
Good contrast → reliable colour features	Low contrast → weak feature maps
Normal resolution → anchors match well	Very low resolution → objects smaller than receptive field
High confidence scores	Low confidence; false negatives increase

---

8. Evaluation Metrics

- **Precision:** TP / (TP + FP) — of all detections, how many are correct?
- **Recall:** TP / (TP + FN) — of all objects, how many were found?
- **mAP (mean Average Precision):** primary metric; average AP across all classes and IoU thresholds
- **FPS (frames per second):** speed metric for real-time systems

---

9. Key Viva Questions — Object Detection

1. **What is Non-Maximum Suppression and why is it needed?** Without NMS, a single object generates many overlapping predictions. NMS keeps the highest-confidence box and removes redundant overlapping boxes above an IoU threshold.

2. **What is the difference between precision and recall in detection?** High precision = few false detections. High recall = few missed objects. There is a trade-off: lowering the confidence threshold increases recall but reduces precision.

3. **Why do detectors struggle with small objects?** Deep layers have small spatial resolution (e.g. 7×7 for a 224×224 input). Small objects map to very few pixels at this scale. Feature Pyramid Networks (FPN) address this by combining multi-scale feature maps.

EXP 7 — Simple RNN (Recurrent Neural Network)

1. Architecture

> 📌 **Definition:** An RNN processes sequential data by maintaining a hidden state that is updated at each time step, allowing information to persist across the sequence.

At each time step t:

- $h = \tanh(W_x \cdot x + W \cdot h_{-1} + b)$ — hidden state update
- $\hat{y} = W_Y \cdot h + b_Y$ — output at time t

Key property: same weights (W_x, W□, W_Y) are shared across all time steps.

---

2. Types of RNN Architectures

Architecture	Input → Output	Example Use Case

One-to-One	Single → Single	Standard classification (not truly RNN)
One-to-Many	Single → Sequence	Image captioning
Many-to-One	Sequence → Single	Sentiment analysis, text classification
Many-to-Many (same length)	Sequence → Sequence	POS tagging, NER
Many-to-Many (encoder-decoder)	Sequence → Sequence (different length)	Machine translation

---

3. Backpropagation Through Time (BPTT)

Gradients flow backward through all time steps. For long sequences, the product of many Jacobians causes:

- **Vanishing Gradients:** gradient → 0; early time steps not updated; network forgets distant past
- **Exploding Gradients:** gradient → ∞; training diverges (mitigated by gradient clipping)

---

4. Limitations of Simple RNN

- Cannot capture long-term dependencies (>10–20 steps effectively)
- Vanishing gradient problem makes training unstable
- Sequential computation — cannot parallelise across time steps
- Performance degrades rapidly as sequence length increases

> 💡 **Key Point:** LSTM and GRU were designed specifically to overcome these limitations via gating mechanisms.

---

5. Effect of Sequence Length

Short Sequences (<20 steps)	Long Sequences (>50 steps)
RNN performs reasonably well	Severe vanishing gradient — context lost

Gradient flows back effectively	Only recent context retained
Stable training	Training unstable; slow convergence
Suitable for simple pattern prediction	LSTM/GRU or Transformer recommended

---

6. Advantages & Disadvantages — Simple RNN

Advantages	Disadvantages
Handles variable-length sequences	Vanishing gradient for long sequences
Parameter sharing across time steps	Cannot capture long-term dependencies
Works for simple sequential patterns	Sequential computation — not parallelisable
Lower parameter count than LSTM/GRU	Performance far below LSTM/GRU on most tasks
Easy to understand and implement	Rarely used in practice for serious tasks

---

7. Key Viva Questions — Simple RNN

1. **What is the vanishing gradient problem in RNNs?** During BPTT, gradients are multiplied by the same weight matrix at each step. If the largest eigenvalue of this matrix < 1, gradients shrink exponentially; the network cannot learn dependencies beyond a few steps.

2. **How is an RNN different from an MLP?** An MLP processes fixed-size input in a single forward pass. An RNN processes sequences step-by-step, maintaining a hidden state that carries information across time steps.

3. **What is gradient clipping?** Capping the gradient norm to a maximum value (e.g. 1.0 or 5.0) before the update step. Prevents exploding gradients from destabilising training.

EXP 8 — Long Short-Term Memory (LSTM)

1. Motivation

📌 **Definition:** LSTM (Hochreiter & Schmidhuber, 1997) addresses the vanishing gradient problem through a cell state (long-term memory) and three learnable gates that control information flow.

---

2. Architecture — The Four Components

Gate / State	Formula	Role
Forget Gate (f_t)	$\sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	Decides what to erase from cell state (0=forget all, 1=keep all)
Input Gate (i_t)	$\sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	Decides which new information to add to cell state
Candidate ($\tilde{C}_t$)	$\tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$	New candidate values to potentially add
Cell State Update	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	Selective update of long-term memory
Output Gate (o_t)	$\sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	Controls what part of cell state becomes hidden state
Hidden State	$h_t = o_t \odot \tanh(C_t)$	Output / short-term memory passed to next step and output layer

💡 **Key Point:** The cell state C_t runs through the entire sequence with only minor linear interactions (forget + add), providing a highway for gradient flow — this is why LSTMs avoid vanishing gradients.

---

3. LSTM vs Simple RNN

Simple RNN	LSTM
Single hidden state h_t	Dual state: cell state C_t + hidden state h_t
No gating mechanism	Three gates (forget, input, output)
Severe vanishing gradient	Cell state provides gradient highway

Short-term memory only	Learns both short and long-term dependencies
~$3N^2$ parameters	~$4N^2$ parameters (4× more than RNN for same hidden size)
Fast training	Slower training; more parameters
Fails beyond ~20 steps	Works well up to hundreds of steps

---

4. Loss Convergence Comparison

On sequence tasks (e.g. text generation, time series):

- **Simple RNN:** loss stagnates early; fails to capture context beyond short window
- **LSTM:** loss decreases consistently; captures long-range patterns
- LSTM validation loss remains closer to training loss (better generalisation)

---

5. Applications of LSTM

- Machine Translation (seq2seq with attention)
- Speech Recognition
- Text Generation and Language Modelling
- Time Series Forecasting (stock prices, weather)
- Video Classification (temporal modelling)
- Anomaly Detection in sequences

---

6. Advantages & Disadvantages — LSTM

Advantages	Disadvantages
Solves vanishing gradient; captures long-range dependencies	4× more parameters than simple RNN
Flexible memory control via gates	Slower to train than GRU
Works on variable-length sequences	Still sequential; hard to parallelise
State-of-the-art on many sequence tasks (pre-Transformer)	Transformers now outperform on most NLP tasks

Robust to noisy sequential data	Complex to debug and interpret

---

7. Key Viva Questions — LSTM

1. **How does the forget gate prevent vanishing gradients?** The forget gate (values 0–1) allows the cell state to persist unchanged (close to 1) across many time steps. Gradients flow back through the cell state without repeated multiplication by weight matrices, avoiding exponential decay.

2. **What is the difference between the cell state and hidden state?** Cell state (C_t): long-term memory; runs through sequence with minimal transformation. Hidden state (h_t): short-term output; filtered version of cell state; passed to next step and to the output layer.

3. **When would you use bidirectional LSTM?** When future context is also available at inference time (e.g. NER, POS tagging, text classification). Not suitable for streaming/real-time prediction where future is unknown.

EXP 9 — Gated Recurrent Unit (GRU)

1. Architecture

📌 **Definition:** GRU (Cho et al., 2014) is a simplified alternative to LSTM with two gates instead of three, merging the cell state and hidden state into a single hidden state.

Gate	Formula	Role
Update Gate (z_t)	$\sigma(W_z \cdot [h_{t-1}, x_t])$	How much of past hidden state to keep (acts like forget+input combined)
Reset Gate (r_t)	$\sigma(W_r \cdot [h_{t-1}, x_t])$	How much of past to forget when computing candidate
Candidate ($\tilde{h}_t$)	$\tanh(W \cdot [r_t \odot h_{t-1}, x_t])$	New candidate hidden state

Hidden State	$h_t = (1-z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	Interpolation between old and new state

---

2. GRU vs LSTM — Detailed Comparison

GRU	LSTM
2 gates (update, reset)	3 gates (forget, input, output)
Single hidden state h_t	Dual state: cell state + hidden state
~$3N^2$ parameters	~$4N^2$ parameters — 33% more
Faster training (fewer ops per step)	Slower; more parameters to update
Comparable accuracy on most tasks	Slight edge on very long sequences
Better for smaller datasets (fewer params → less overfit)	Better when very long memory needed
Simpler implementation	More expressive in theory

---

3. When to Choose GRU over LSTM

- When training time and computational budget are limited
- When dataset is small to medium (GRU has fewer parameters → less overfitting)
- Real-time / edge applications where inference speed matters
- When sequence lengths are moderate (not extremely long)

> 💡 **Key Point:** In practice, GRU and LSTM perform similarly on most tasks. Prefer GRU for efficiency; LSTM if long-term memory is critical and compute is available.

---

4. Efficiency & Accuracy Trade-offs

Metric	RNN	GRU	LSTM
Training Time	Fastest	Fast	Slowest

Parameters	Fewest	Medium	Most
Long-term Memory	Poor	Good	Best
Short Sequences	Good	Good	Good
Long Sequences	Fails	Good	Best
Inference Speed	Fastest	Fast	Moderate
Implementation Complexity	Simple	Medium	Complex

---

5. GRU for Real-Time Applications

- Fewer gates = fewer matrix multiplications per step → lower latency
- Can be deployed on mobile/edge with INT8 quantisation
- Used in keyword spotting, online time-series anomaly detection
- Bidirectional GRU offers strong accuracy with less compute than BiLSTM

---

6. Advantages & Disadvantages — GRU

Advantages	Disadvantages
Faster training than LSTM	Slightly less expressive than LSTM on very long sequences
Fewer parameters; less prone to overfitting	Still sequential; cannot parallelise across time
Simpler to implement and tune	Transformers outperform on NLP with attention
Good accuracy on most sequence tasks	No explicit cell state — less control over memory
Suitable for real-time / resource-constrained systems	Not the state-of-the-art for most large-scale tasks

---

7. Key Viva Questions — GRU

1. **How does the update gate in GRU combine the functionality of LSTM's forget and input gates?** Update gate z_t close to 1 → mostly use candidate state (input gate open). z_t close to 0 → keep old hidden state (forget gate closed). One gate jointly controls forgetting old info and adding new info.

2. **Why is GRU preferred in real-time applications?** Fewer gating operations per time step → lower latency. Fewer parameters → smaller model size. Comparable accuracy to LSTM for most tasks justifies the trade-off.

3. **What is the reset gate's role?** The reset gate r_t controls how much of the previous hidden state is used when computing the candidate. $r_t \approx 0$ means the candidate ignores the past completely (useful for resetting context at sentence boundaries).

EXP 10 — Generative Adversarial Network (GAN)

1. Architecture

> 📌 **Definition:** A GAN (Goodfellow et al., 2014) consists of two neural networks trained in opposition: a Generator that creates fake samples and a Discriminator that distinguishes real from fake.

Two-player minimax game:

- **Generator G:** maps random noise $z \sim N(0,I)$ to synthetic data $G(z)$
- **Discriminator D:** outputs probability $D(x)$ that input x is real
- **Objective:** `min_G max_D E[log D(x)] + E[log(1 - D(G(z)))]`

---

2. Training Procedure

1. Sample random noise z from latent space (e.g. 100-d Gaussian vector)
2. G generates fake image G(z)
3. D receives real image x (label=1) and fake G(z) (label=0)
4. Compute discriminator loss and update D to classify correctly
5. Compute generator loss = −log D(G(z)) and update G to fool D
6. Alternate steps until convergence (or training ends)

---

3. Architecture Details

Generator	Discriminator
Input: random noise vector (e.g. 100-d)	Input: real or generated image
Architecture: Dense → Reshape → Upsample (Transposed Conv)	Architecture: Conv → BN → LeakyReLU → Flatten → Dense
Activations: ReLU (hidden), Tanh (output)	Activations: LeakyReLU (hidden), Sigmoid (output)
Output: synthetic image (same shape as real data)	Output: scalar probability [0,1]
Loss: wants D to output 1 for its fake images	Loss: wants to correctly classify real=1, fake=0

---

4. Training Progression

How generated outputs improve over training:

- **Early epochs:** random noise / blurry uniform patches
- **Mid training:** rough shapes emerge; colour distributions improve
- **Later epochs:** realistic textures, object structure visible
- **Convergence:** D outputs ~0.5 for all inputs (cannot distinguish real/fake)

> ⚠️ **Note:** If D becomes too strong early, generator gradients vanish (discriminator is 'too good'). If G becomes too strong, D collapses. Balancing training is critical.

---

5. Generator vs Discriminator Training Dynamics

Generator	Discriminator
Wants D loss to be high (fool D)	Wants to minimise its own classification error
Loss: binary cross-entropy of fake samples predicted as real	Loss: binary cross-entropy on real + fake samples

If D is perfect, G gradients vanish → training stalls	If D is too weak, G never improves properly
Good G: produces diverse realistic images	Good D: confidently separates real from fake
Training difficulty: mode collapse (generates same image)	Training difficulty: memorising real data

---

6. Common GAN Problems & Solutions

Problem	Description	Solution
Mode Collapse	G generates same output regardless of z	Minibatch discrimination; Wasserstein GAN
Training Instability	Losses oscillate; training diverges	Spectral normalisation; gradient penalty; careful LR
Vanishing Gradient	D too accurate; G gradient ≈ 0	Feature matching; label smoothing
Checkerboard Artifacts	Upsample + Conv causes grid patterns	Use bilinear upsample instead of transposed conv
Evaluation Difficulty	No single loss tells quality	FID score; Inception Score; visual inspection

---

7. GAN Variants

GAN Variant	Key Feature	Use Case
DCGAN	Deep convolutional G and D	Stable training for image generation
CGAN (Conditional)	G and D conditioned on class label y	Class-controlled generation
WGAN	Wasserstein distance as loss; weight clipping	Training stability; meaningful loss metric
CycleGAN	Two G + two D; cycle consistency loss	Unpaired image-to-image translation

StyleGAN	Style-based generator; AdaIN normalisation	High-quality face synthesis (FFHQ)
Pix2Pix	Paired image translation with L1 + GAN loss	Image-to-image (sketch→photo, map→aerial)

---

8. Evaluating Generated Sample Quality

- **FID (Fréchet Inception Distance):** measures similarity of real and generated feature distributions. Lower = better.
- **Inception Score (IS):** measures quality + diversity. Higher = better.
- **Precision & Recall:** precision = fidelity; recall = diversity
- **Visual inspection:** most used in practice for early debugging

---

9. Advantages & Disadvantages — GAN

Advantages	Disadvantages
Generates highly realistic data (images, audio, video)	Training is unstable; requires careful tuning
No explicit likelihood needed (implicit model)	Mode collapse: G may generate limited variety
Powerful for data augmentation and synthesis	Evaluation metrics are indirect and complex
Wide range of variants for specific tasks	Computationally expensive; needs GPU
Creative applications: style transfer, super-resolution	No direct control over latent space without CGAN/VAE

---

10. Key Viva Questions — GAN

1. **What is mode collapse and how does it occur?** The generator discovers a small set of outputs that consistently fool the discriminator and only generates those, ignoring diversity. It occurs when the discriminator's gradient signal is too uniform. WGAN and minibatch discrimination help prevent it.

2. **Why is GAN training a minimax game?** The generator minimises (wants D to be wrong) while the discriminator maximises (wants to be correct). They are in direct opposition, and Nash equilibrium is reached when neither can improve unilaterally — D outputs 0.5 for all inputs.

3. **What does it mean when discriminator loss = 0.5?** The discriminator cannot distinguish real from fake — it is completely confused. This is the ideal convergence point where the generator has learned to produce perfectly realistic samples.

4. **How is FID score calculated?** Features from real and generated images are extracted using InceptionV3. Mean and covariance of both distributions are compared. FID = $||\mu_r - \mu_g||^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{0.5})$. Lower FID = distributions are closer = better quality.

Master Comparison — All Models

Architecture & Capability Overview

Model	Input Type	Key Mechanism	Long-range Dep.	Params	Best For
MLP	Fixed vector	Fully connected layers	No	Medium	Tabular, digit classification
CNN	Grid (images)	Conv filters + pooling	Limited (local)	Medium–Large	Image classification, vision
Pretrained CNN (TL)	Images	Pretrained conv features	Limited	Large (frozen)	Small dataset vision tasks
Simple RNN	Sequence	Hidden state recurrence	No (vanishes)	Small	Very short sequences
LSTM	Sequence	Cell state + 3 gates	Yes	Large	Long sequences, NLP, TS
GRU	Sequence	Hidden state + 2 gates	Yes	Medium	Fast sequence modelling

GAN (G)	Noise vector	Adversarial generation	—	Medium–Large	Data synthesis, augmentation

---

When to Use Which Model

Task	Recommended Model	Why
Tabular / structured data	MLP	Simple, effective; no spatial structure needed
Image classification (large dataset)	CNN from scratch	Full control; spatial feature learning
Image classification (small dataset)	Transfer Learning (ResNet/VGG)	Pretrained features; avoids overfitting
Object detection	YOLOv8 / Faster R-CNN (pretrained)	Specialised architectures; state-of-the-art
Sentiment analysis / text classification	LSTM or GRU (or Transformer)	Handles variable-length text sequences
Time series forecasting	LSTM / GRU	Temporal dependencies captured
Machine translation	LSTM seq2seq + Attention	Long sequences with complex dependencies
Real-time sequence prediction	GRU	Faster than LSTM; good accuracy
Generating synthetic images	GAN (DCGAN / StyleGAN)	Adversarial training produces realistic outputs
Few-shot learning	Transfer Learning (fine-tune)	Pretrained knowledge + small task data

---

Gradient Problems — Summary

Problem	Solution
Vanishing gradient (deep MLP)	ReLU activation, Batch Normalisation, residual connections

Vanishing gradient (long RNN)	LSTM / GRU gating mechanisms
Exploding gradient (RNN training)	Gradient clipping (clip norm to max value)
Dying ReLU	Leaky ReLU, ELU, or proper weight initialisation
GAN vanishing gradient	Feature matching, label smoothing, WGAN

Regularisation Techniques — When to Apply

Technique	Applicable To	Effect
Dropout	MLP, CNN, LSTM	Randomly zeroes activations during training; reduces co-adaptation
Batch Normalisation	MLP, CNN	Normalises layer inputs; stabilises training; faster convergence
L2 Regularisation (Weight Decay)	All	Penalises large weights; prevents overfitting
Data Augmentation	CNN, TL	Increases effective dataset size; improves generalisation
Early Stopping	All	Halts when val loss stops improving; prevents overfit
Max Pooling	CNN	Reduces spatial dimensions; implicit regulariser
Gradient Clipping	RNN, LSTM, GRU	Prevents exploding gradients in recurrent training
Label Smoothing	GAN, Classification	Prevents discriminator/classifier overconfidence

---

Quick Reference Glossary

Term	Definition
Epoch	One full pass through the entire training dataset

Batch Size	Number of samples per gradient update
Learning Rate (η)	Step size for weight update; controls convergence speed
Overfitting	Model memorises training data; poor generalisation to unseen data
Underfitting	Model too simple to capture pattern; high train and val error
Backpropagation	Algorithm to compute gradients via chain rule through the network
Activation Function	Non-linear function applied after linear transformation (ReLU, Sigmoid, etc.)
Feature Map	Output of a convolutional filter applied to input
Receptive Field	Region in the original input that influences a neuron's activation
Pooling	Downsampling operation (Max, Average) to reduce spatial dimensions
Transfer Learning	Reusing a pretrained model's weights for a new task
Fine-tuning	Allowing pretrained weights to update on new task data (small LR)
Vanishing Gradient	Gradients shrink to near zero in deep networks; early layers don't learn
Exploding Gradient	Gradients grow exponentially; training diverges
BPTT	Backpropagation Through Time — gradient computation in RNNs
Hidden State (h_t)	RNN/LSTM/GRU memory vector passed between time steps
Cell State (C_t)	LSTM's long-term memory; persists with minimal transformation
Gate (LSTM/GRU)	Sigmoid-gated mechanism controlling information flow (0=block, 1=pass)
Mode Collapse (GAN)	Generator produces limited variety of outputs
FID Score	Fréchet Inception Distance — measures quality of GAN outputs (lower=better)
mAP	Mean Average Precision — primary metric for object detection models
IoU	Intersection over Union — measures overlap between predicted and true bounding box

NMS	Non-Maximum Suppression — removes redundant overlapping detections
Dropout	Randomly zeroes neurons during training to reduce co-adaptation
Batch Normalisation	Normalises layer inputs to zero mean, unit variance per mini-batch
Data Augmentation	Artificial dataset expansion via transforms (flip, crop, rotate)
One-hot Encoding	Represents class label as binary vector (e.g. class 3 of 5 → [0,0,1,0,0])
Softmax	Output activation converting logits to class probabilities summing to 1
Cross-Entropy Loss	Standard loss for classification: $L = -\Sigma\, y \cdot \log(\hat{y})$
Confusion Matrix	Table showing TP, FP, TN, FN for each class